OPERATING SYSTEMS (UE25CS242A)

LEVEL 3 MINI-PROJECT

CampusCompute: Intelligent Job Scheduling and Resource Management

1. Scenario

A university operates a shared computing lab where students and researchers run CPU-intensive programs and simulations. During peak hours, jobs compete for CPU time and shared resources. The administrator needs better job scheduling, synchronized activity logging, inter-process communication, resource-safety checks, and worker-failure detection. Design and implement CampusCompute, a command-line prototype demonstrating these operating-system concepts.

Implement in C on Linux/Ubuntu or WSL Ubuntu. The output must be generated by working code; a menu-only mock-up or hard-coded results are not sufficient.

2. Functional Requirements

- **FR1 – Job management and process lifecycle:** Represent jobs using fields such as job ID, arrival time, CPU burst, priority, and status. Validate input and track execution/completion.
- **FR2 – CPU scheduling:** Implement FCFS, Round Robin, and non-preemptive Priority Scheduling. Display execution order/timeline, completion time, turnaround time, waiting time, and averages. State the tie-breaking rule and Round Robin time quantum.
- **FR3 – Concurrent activity logger:** Use POSIX threads to implement a bounded producer-consumer queue. Protect shared data with a mutex and coordinate full/empty conditions using condition variables or a justified alternative. Write readable event logs and terminate threads cleanly.
- **FR4 – Inter-process communication:** Use POSIX shared memory to exchange data between cooperating processes and a named semaphore or suitable POSIX synchronization mechanism. Handle errors and clean up resources created by the program.
- **FR5 – Resource-safety analysis:** Implement the Banker's safety algorithm using Available, Allocation, Maximum, and Need (Need = Maximum − Allocation). Show a safe sequence for a safe state and identify an unsafe state. Test both cases.
- **FR6 – Worker failure detection:** Create child processes using fork() and collect status with waitpid() or an equivalent documented mechanism. Demonstrate normal completion and a controlled simulated failure; report the result accurately.
- **FR7 – Resource limits and cleanup:** Where supported, demonstrate at least one process resource limit using setrlimit() or a suitable Linux mechanism. Explain its observed effect. Clean up threads, processes, files, shared memory, and semaphores created by the program.
- **FR8 – Command-line interface and error handling:** Provide documented commands or a menu. Invalid commands and malformed input must produce a useful error/usage message. Each implemented feature must be independently testable.

3. Technical Requirements and Rules

- Use C on Linux/Ubuntu or WSL Ubuntu.

- Git/GitHub is not required and must not be used as a submission requirement.
- Submit source code and a Makefile or exact build instructions. The program must build from source.
- Use meaningful function names, comments for non-obvious logic, input validation, and error handling.
- Do not hard-code output instead of implementing the required algorithm. Use repeatable test inputs where possible.
- Clearly identify simulated worker failures; do not present them as real OS failures.
- Do not remove unrelated files or system-wide IPC objects. Clean up only resources created by your program.
- Every terminal screenshot must show the student's complete, actual SRN in the terminal prompt. Printing the SRN inside a file is not sufficient.
- Before capturing screenshots, configure the prompt with your own full SRN. Example: export SRN='YOUR_FULL_SRN'; export PS1='[$SRN] \w\$ '. Replace the example with your actual SRN.
- Use genuine screenshots from your own terminal. Do not edit output, fabricate results, or use another student's screenshots.
- Number screenshots S01, S02, S03, etc. Each screenshot must have one short sentence explaining what it demonstrates. Do not reuse screenshot numbers.
- Do not include a separate project report. The required evidence is the screenshots with one-line explanations.
- Combine the screenshots and their one-line explanations into a single PDF. Name the PDF using your own SRN only, for example PES1UGXXXXXXX.pdf. Do not use the example SRN.

4. Screenshot Checklist

Capture the applicable screenshots below. Each screenshot must include the command and meaningful output, with the complete SRN visible in the terminal prompt. Add one short explanatory sentence under each screenshot in the PDF. If a feature is not implemented or is not applicable, do not fabricate evidence.

- **S01 – Environment and SRN prompt:** Shows Ubuntu/WSL terminal, full SRN prompt, and working directory.
- **S02 – Compiler and build tools:** Shows GCC and Make versions or confirms the required tools are available.
- **S03 – Project files:** Shows the project directory, source files, and Makefile.
- **S04 – Clean build:** Shows the program compiling from source without unresolved compiler errors.
- **S05 – CLI help/usage:** Shows available commands or menu options.
- **S06 – FCFS scheduling:** Shows FCFS execution order and calculated scheduling metrics.
- **S07 – Round Robin scheduling:** Shows the time quantum, execution order, and scheduling metrics.
- **S08 – Priority scheduling:** Shows job priorities, tie-breaking rule where needed, and scheduling metrics.
- **S09 – Scheduling comparison:** Shows a comparison of scheduling policies using the same job set.
- **S10 – Producer-consumer logger:** Shows producer/consumer activity and generated log entries.
- **S11 – Thread synchronization and termination:** Shows synchronized operation and orderly thread completion.
- **S12 – Shared-memory IPC:** Shows cooperating processes exchanging data through shared memory.

- **S13 – Semaphore coordination and cleanup:** Shows synchronization using the implemented semaphore and cleanup of created IPC resources.
- **S14 – Banker's safe state:** Shows a safe resource state and a valid safe sequence.
- **S15 – Banker's unsafe state:** Shows the algorithm identifying an unsafe resource state.
- **S16 – Normal worker process:** Shows child-process creation and normal completion status.
- **S17 – Simulated worker failure:** Shows controlled non-zero/abnormal worker termination and parent-side detection.
- **S18 – Resource-limit demonstration:** Shows the resource limit applied and the observed effect.
- **S19 – Invalid input/command:** Shows an invalid command or input and the resulting error/usage message.
- **S20 – Cleanup verification:** Shows that created processes/threads have ended and program-created IPC resources are released.
- **S21 – Integrated execution :** Shows the integrated demonstration and its summary.
- **S22 – Generated logs:** Shows the log file created by the program and a sample of its contents.
- **S23 – Final source and build files:** Shows the final source files, Makefile, and executable.
- **S24 – Submission ZIP :** Shows the archive contents and a successful archive integrity check.

5. Submission

- Source code and required header/configuration files.
- Makefile or exact build instructions, plus a README with prerequisites and run/cleanup commands.
- One PDF containing the numbered screenshots S01 onward, each followed by a one-line explanation. The PDF filename must be the student's own SRN only (SRN.pdf).
- Submit a ZIP containing the source code and build files if required by the course submission portal. Do not include a separate report.